

UNIT: 8

APPLETS AND APPLICATION

8.1 Fundamentals concept of Applet:

Introduction:

- Applets are small java programs that are used in internet computing.
- They can be transported over the internet from one computer to another computer and run using the AppletViewer or any web browser that supports java.
- Applet can be used to perform arithmetic operations, display graphics, play sounds, accept user input, create animation and play interactive games like any other application.
- The Applets are used to provide interactive features to web applications that cannot be provided by HTML alone.
- Applets do not need a main () method.
- Applets must be run under an applet viewer or a Java-compatible browser.
- User I/O is not accomplished with Java's stream I/O classes. Instead, applets use the interface provided by the AWT or Swing.
- All applets are subclasses (either directly or indirectly) of Applet.

Types of applet:

There are two types of applet. They are

1. **Applet Based on Abstract Window Toolkit (AWT):**
 - It is used to provide the graphical user interface (or use no GUI at all).
 - This style of applet has been available since Java was first created.
2. **Applets based on the Swing class JApplet:**
 - They are the ones which inherits Applet.
 - Swing applets use the Swing classes to provide the GUI. Swing offers a richer and often easier-to-use user interface than does the AWT.
 - Thus, Swing-based applets are now the most popular.
 - However, traditional AWT-based applets are still used, especially when only a very simple user interface is required.

Thus, both AWT- and Swing-based applets are valid.

8.2 Simple Applet and Applet and Application:

Simple applet look like:

```
import java.applet.*;

import java.awt.*;

public class HelloWorldApplet extends Applet {

    public void paint (Graphics g) {

        g.drawString ("Hello World", 25, 50);

    }

}
```

When to use applet

- When we need something dynamic to be included in the display of a web page.
- When we require some flash output.
- When we want to create a program and make it available on the internet for use by others on their computers

Steps used to create an Applet

1. Building an applet code(.java file)
2. Creating an executable applet(.class file)
3. Designing a web page using HTML tags
4. Preparing an <APPLET> tag into the web page
5. Incorporating <APPLET> tag into the web page
6. Creating HTML file
7. Testing the applet code

Difference between applet and application

Features	Application	Applet
main () method	Present	Not present
Execution	Requires JRE	Requires a browser or applet viewer
Nature	Called as stand-alone application as application can be executed from command prompt	Requires some third-party tool help like a browser to execute
Restrictions	Can access any data or software available on the system	cannot access anything on the system except browser's services
Security	Does not require any security	Requires highest security for the system as they are untrusted

8.3 Applet architecture/Life cycle, Parameters to Applet:

Applet Architecture:

- An applet is a window-based program. First, applets are event driven.
- An applet resembles a set of interrupt service routines.

Here is how the process works.

- An applet waits until an event occurs.
- The AWT notifies the applet about an event by calling an event handler that has been provided by the applet.
- Once this happens, the applet must take appropriate action and then quickly return control to the AWT.
- This is a crucial point. For the most part, your applet should not enter a “mode” of operation in which it maintains control for an extended period.
- Instead, it must perform specific actions in response to events and then return control to the AWT run-time system.

Second, the user initiates interaction with an applet—not the other way around.

- In a no windowed program, when the program needs input, it will prompt the user and then call some input method, such as `readLine ( )`.
- This is not the way it works in an applet. Instead, the user interacts with the applet as he or she wants, when he or she wants.
- These interactions are sent to the applet as events to which the applet must respond.
- For example, when the user clicks a mouse inside the applet's window, a mouse-clicked event is generated.
- If the user presses a key while the applet's window has input focus, a keypress event is generated.
- As you will see in later chapters, applets can contain various controls, such as push buttons and check boxes.
- When the user interacts with one of these controls, an event is generated.

Applet Life Cycle:

There are five applet methods that are called by the applet container from the time the applet is loaded into the browser to the time it's terminated by the browser. These methods correspond to various aspects of an applet's life cycle.

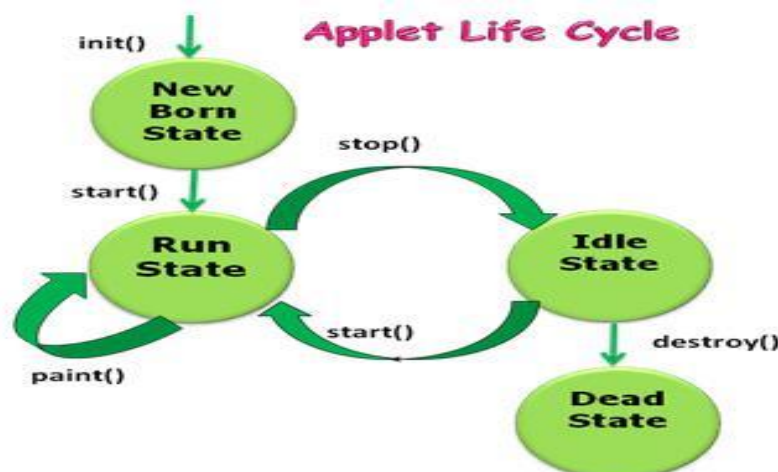


Fig: Applet Life Cycle

Every Applet has:

1. **Born state**
2. **Running state**
3. **Idle state (may or may not)**
4. **Dead state**

1. Born State

When the class is executed, the `init()` function is automatically invoked. At this stage an Applet is said to be born. This state is also called initialization state. During the applet life cycle, the born state occurs only once. The `init()` function is defined within an Applet. This function is used to initialize instance members, to load the required fonts, to initialize objects, to load pictures, to load back colors, etc.

2. Running State

After initialization, this state will automatically occur by invoking the start() method of applet class which again calls the run() method and which calls the paint() method. The running state also occurs from idle state when the applet is reloaded.

3. Idle State

The idle state will make the execution of the applet to be halted temporarily. Applet moves to this state when the currently executed applet is minimized or when the user switches over to another page. At this point the stop() method is invoked. From the idle state the applet can move to the running state.

4. Dead State

When the applet programs terminate, the destroy function is invoked which makes an applet to be in dead state.

❖ Advantages of Applet:

- As applet is a small java program, so it is platform independent code which is capable to run on any browser.
- Applets can perform various small tasks on client-side machines. They can play sounds, show images, get user inputs, get mouse clicks and even get user keystrokes, etc .
- Applets creates and edit graphics on client-side which are different and independent of client-side platform.
- As compared to stand-alone application applets are small in size, the advantage of transferring it over network makes it more usable.
- Applets are quite secure because of their access to resources.
- Applets are secure and safe to use because they cannot perform any modifications over local system.
- Various small tasks such as performing login, inventory checking, task scheduling can be done by applets running over Intranets.

8.4. The HTML APPLET Tag:

The APPLET tag be used to start an applet from both an HTML document and from an applet viewer.

CODEBASE is an optional attribute that specifies the base URL of the applet code, which is the directory that will be searched for the applet's executable class file. The HTML document's URL directory is used as the CODEBASE if this attribute is not specified. The CODEBASE does not have to be on the host from which the HTML document was read.

CODE is a required attribute that gives the name of the file containing applet's compiled .class file. This file is relative to the code base URL of the applet, which is the directory that the HTML file was in or the directory indicated by CODEBASE if set.

ALT The ALT tag is an optional attribute used to specify a short text message that should be displayed if the browser recognizes the APPLET tag but can't currently run Java applets.

WIDTH and **HEIGHT** are required attributes that give the size (in pixels) of the applet display area.

NAME is an optional attribute used to specify a name for the applet instance. Applets must be named in order for other applets on the same page to find them by name and communicate with them. To obtain an applet by name, use ***getApplet()***, which is defined by the ***AppletContext*** interface.

ALIGN is an optional attribute that specifies the alignment of the applet. This attribute is treated the same as the HTML IMG tag with these possible values: LEFT, RIGHT, TOP, BOTTOM, MIDDLE, BASELINE, TEXTTOP, ABSMIDDLE, and ABSBOTTOM.

VSPACE and **HSPACE** are optional attributes. VSPACE specifies the space, in pixels, above and below the applet. HSPACE specifies the space, in pixels, on each side of the applet. They're treated the same as the IMG tag's VSPACE and HSPACE attributes.

PARAM NAME and **VALUE** The PARAM tag allows us to specify applet-specific arguments in an HTML page. Applets access their attributes with the ***getParameter()*** method.

❖ Passing Parameters to applet:

The **APPLET tag** in HTML allows us to pass parameters to our applet. To retrieve a parameter, we need to use the ***getParameter()*** method. It returns the value of the specified parameter in the form of a ***String*** object. Thus, for ***numeric*** and ***boolean*** values, we will need to convert their string representations into their internal formats.

Use Parameters

```
/*
<applet code="ParamDemo" width="400" height="100"> <param name="fontName"
value="Times new Roman"> <param name="fontSize" value="15">
<param name="leading" value="3">
<param name="accountEnabled" value="true"> </applet>
*/
import java.awt.*;
import java.applet.*;
public class ParamDemo extends Applet{
String fontName;
int fontSize;
float leading;
boolean active;
Initialize the string to be displayed. public void start() {
String param;
fontName = getParameter("fontName"); if(fontName == null)
fontName = "Not Found";
param = getParameter("fontSize");
try {
if(param != null) // if not found
fontSize = Integer.parseInt(param);
else
fontSize = 0;

} catch(NumberFormatException e) { fontSize = -1;
}
}
```

```

param = getParameter("leading"); try {
if(param != null) // if not found
leading = Float.valueOf(param).floatValue(); else
leading = 0;
} catch(NumberFormatException e) {
leading = -1;
}
param = getParameter("accountEnabled"); if(param != null)
active = Boolean.valueOf(param).booleanValue(); }
// Display parameters.
public void paint(Graphics g) {
g.drawString("Font name: " + fontName, 0, 10);
g.drawString("Font size: " + fontSize, 0, 26);
g.drawString("Leading: " + leading, 0, 42);
g.drawString("Account Active: " + active, 0, 58);
}
}

```

8.5 The Applet Class:

Methods

void destroy()

AccessibleContext

getAccessibleContext()

AppletContext

getAppletContext()

String getAppletInfo()

AudioClip

getAudioClip(URL url)

URL getCodeBase()

Image getImage(URL url)

Locale getLocale()

*String getParameter(String
paramName)*

void init()

boolean isActive()

void play(URL url)

*void play(URL url, String
clipName)*

void start()

void stop()

void showStatus(String str)

And many more.. (refer to text book)

Description

Called by the browser just before an applet is terminated.

Returns the accessibility context for the invoking object.

Returns the context associated with the applet.

Returns a string that describes the applet.

Returns an AudioClip object that encapsulates the audio clip found at the location specified by url.

Returns the URL associated with the invoking applet.

Returns an Image object that encapsulates the image found at the location specified by url.

Returns a Locale object that is used by various locale sensitive classes and methods.

Returns the parameter associated with *paramName*. *null* is returned if the specified parameter is not found.

Called when an applet begins execution. It is the first method called for any applet.

Returns true if the applet has been started. It returns false if the applet has been stopped.

If an audio clip is found at the location specified by url, the clip is played.

If an audio clip is found at the location specified by url with the name specified by clipName, the clip is played.

Called by the browser when an applet should start (or resume) execution. It is automatically called after *init()* when an applet first begins.

Called by the browser to suspend execution of the applet. Once stopped, an applet is restarted when the browser calls *start()*.

Displays str in the status window of the browser or applet viewer. If the browser does not support a status window, then no action takes place.

Table: The Methods Defined by Applet

8.6 Applet Security Policies:

- Because applets are designed to be loaded from a remote site and then executed locally, security becomes vital.
- The user never gets a chance to confirm or to stop individual applets from running.
- For this reason, applets (unlike applications) are restricted in what they can do.
- The applet security manager throws a **SecurityException** whenever an applet attempts to violate one of the access rules.
- The restricted execution environment for applets is often called the “sandbox.” Applets playing in the “sandbox” cannot alter the user's system or spy on it.

In particular, when running in the sandbox:

- ✓ Applets can never run any local executable program.
- ✓ Applets cannot communicate with any host other than the server from which they were downloaded; that server is called the **originating host**.
- ✓ This rule is often called “applets can only phone home.”
- ✓ This protects applet users from applets that might try to spy on intranet resources.
- ✓ Applets cannot read from or write to the local computer's file system.
- ✓ Applets cannot find out any information about the local computer, except for the Java version used, the name and version of the operating system, and the characters used to separate files (for instance, / or \), paths (such as : or ;), and lines (such as \n or \r\n).

To sum up, Java has three separate mechanisms for enforcing security:

- ✓ Program code is interpreted by the Java Virtual Machine, not executed directly.
- ✓ A security manager checks all sensitive operations in the Java runtime library.
- ✓ Applets can be signed to identify their origin.

Example of displaying image in applet:

```
import java.awt.*;

import java.applet.*;

public class DisplayImage extends Applet {

    Image picture;

    public void init() {

        picture = getImage(getDocumentBase(),"name of picture.jpg");

    }

    public void paint(Graphics g) {

        g.drawImage(picture, 30,30, this);

    } }
```

In the above example, drawImage() method of Graphics class is used to display the image. The 4th argument of drawImage() method of is ImageObserver object. The Component class implements ImageObserver interface. So current class object would also be treated as ImageObserver because Applet class indirectly extends the Component class.

myapplet.html

```
<html>

<body>

<applet code="DisplayImage.class" width="300" height="300">

</applet>

</body>

</html>
```

To convert from java application to applet and vice versa with example program:**To convert from a Java Applet to an application:**

- Delete import java.applet.*;.Alter the class header so that it extends Frame rather than Applet
- Change the name of the init method to the name of the class, making the former init method the constructor for the class. Delete the word void in the declaration since constructors have no return type .
- Add an invocation of a method to set the title and the layout of the frame by adding the lines setTitle(<String>) and setLayout(new FlowLayout()); inside the constructor .
- Declare and add a button to close the window or implement WindowListener (not shown, but see reference books cited above).
- Create a new method called main, with the header public static void main (String[] args). This method should create a Frame object and should use the methods setSize(<int>,<int>) and setVisible(true) to display the frame.

Example Program:

```
import java.awt.*;
import java.awt.event.*
public class BalloonFrame extends Frame
implements ActionListener {
    private Button grow, shrink, left, right, close;
    private Balloon theBalloon;
    public BalloonFrame() {
        setTitle("Balloon");
        setLayout(new FlowLayout());
        grow = new Button("Grow");
        add(grow);  grow.addActionListener(this);
        shrink = new Button("Shrink");
        add(shrink);
        shrink.addActionListener(this);
        left = new Button("Left");
        add(left);
        left.addActionListener(this);
        right = new Button("Right");
        add(right);
        right.addActionListener(this);
        close = new Button("Exit");
```



```
        add(close);
        close.addActionListener(this);
        theBalloon = new Balloon();
    }
    public static void main(String args[]) {
        BalloonFrame f = new BalloonFrame();
        f.setSize(500, 500);
        f.setVisible(true);
    }
    public void actionPerformed(ActionEvent event) {
        if (event.getSource() == grow)
            theBalloon.grow();
        if (event.getSource() == shrink)
            theBalloon.shrink();
        if (event.getSource() == left)
            theBalloon.moveLeft();
        if (event.getSource() == right)
            theBalloon.moveRight();
        if (event.getSource() == close)
            System.exit(0);
        repaint();
    }
    public void paint(Graphics g) {
        theBalloon.display(g);
    }
}
```

To convert from a Java application into an applet:

Create an HTML file with an applet tag to invoke the applet.

For example, here is the HTML file for the Balloon example:

```
<html>
<body>
<applet code="BalloonApplet.class" width=400 height=200>
</applet>
</body>
</html>
```

➤ Add the line `import java.applet.*;` at the top (line 1 in `BalloonApplet.java`, below).
Alter the class header so that it extends `Applet` rather than `Frame` (line 5).

- Delete the `main` method.
- Change the name of the constructor method from the name of the class to `init` (line 10).

Note that there is no need to create a button to close the window as in the `Frames` example, above. Any such button and its associated action in the `actionPerformed` method can be deleted.

For example, here is the entire `BalloonApplet.java` file:

```
1 import java.applet.*;
2 import java.awt.*;
3 import java.awt.event.*;
5 public class BalloonApplet extends Applet implements ActionListener {
7     private Button grow, shrink, left, right;
8     private Balloon theBalloon;
10    public void init() {
11        grow = new Button("Grow");
12        add(grow);
13        grow.addActionListener(this);
15        shrink = new Button("Shrink");
16        add(shrink);
17        shrink.addActionListener(this);
19        left = new Button("Left");
20        add(left);
21        left.addActionListener(this);
23        right = new Button("Right");
24        add(right);
25        right.addActionListener(this);
27        theBalloon = new Balloon();
28    }
30    public void actionPerformed(ActionEvent event) {
31        if (event.getSource() == grow)
32            theBalloon.grow();
33        if (event.getSource() == shrink)
34            theBalloon.shrink();
35        if (event.getSource() == left)
36            theBalloon.moveLeft();
37        if (event.getSource() == right)
38            theBalloon.moveRight();
39        repaint();
40    }
42    public void paint(Graphics g) {
43        theBalloon.display(g);
44    }
45 }
```

Balloon.java file used by both of the above programs:

```
1 import java.awt.*;
3 public class Balloon {
5     private int diameter;
6     private int x, y;
8     public Balloon() {
9         diameter = 10;
10        x = 20;
11        y = 50;
12    }
14    public void display(Graphics g) {
15        g.drawOval(x, y, diameter, diameter);
16    }
18    public void moveLeft() {
```

```
19    x -= 10;
20  }
22  public void moveRight() {
23    x += 10;
24  }
26  public void grow() {
27    diameter += 5;
28  }
30  public void shrink() {
31    diameter -= 5;
32  }
33 }
```

Write an applet program that demonstrate the message “Welcome to Applet programming”.

```
import java.awt.*;
import java.applet.*;
public class message extends Applet
{
    String msg;
    public void init()
    {
        msg = “Welcome to applet programming”;
    }
    public void paint(Graphics g)
    {
        g.drawString(msg,10,30);
    }
}
```

Save this file with name message.java

Compile this file

Then create this html file

```
<html>
<head>
<title> This is an Applet</title>
<body>
<applet code = message.class height = 100 width = 400> </applet>
</body>
</html>
```

- Save this file with name message.html
 - Run this file as
 - appletviewer message.html
- from the command prompt.

Program that print the sum of two values using applet

```
import java.awt.*;
import java.applet.*;
public class AppletDemo extends Applet
{
    int x,y,z;
```

```
String sum;
public void init()
{
    x = 10;
    y = 20;
    z = x + y;
    sum = "The sum is " +String.valueOf(z);
    setBackground(Color.red);
}
public void paint(Graphics g)
{
    g.drawString(sum,10,30);
}
}

<html>
<applet code = AppletDemo.class width = 300 height = 400>
</applet>
</html>
```

For drawing a human face (eyes – black, ears – pink, face – yellow, lips – red)

```
import java.awt.*;
import java.applet.*;
public class Face extends Applet
{
    public void paint(Graphics g)
    {
        g.setColor(Color.yellow);
        g.fillOval(150,150,100,100);
        g.setColor(Color.black);
        g.fillOval(170,170,15,15);
        g.fillOval(215,170,15,15);
        g.drawLine(200,190,200,210);
        g.setColor(Color.red);
        g.fillOval(190,220,20,10);
        g.setColor(Color.pink);
        g.fillOval(135,185,15,25);
        g.fillOval(250,185,15,25);
    }
}

<html>
<applet code = Face.class width = 500 height = 300>
</applet>
</html>
```

Important Questions from this unit:

- Q.1) Define applet? Explain the life cycle of applet with sample example.
- Q.2) how does applet differ from application? Explain different HTML tags in Applet.
- Q.3) what are parameters to applet? Explain the security policies of applet.
- Q.4) Write an example code to differentiate applet and application.
- Q.5) convert java applet to application and vice versa.